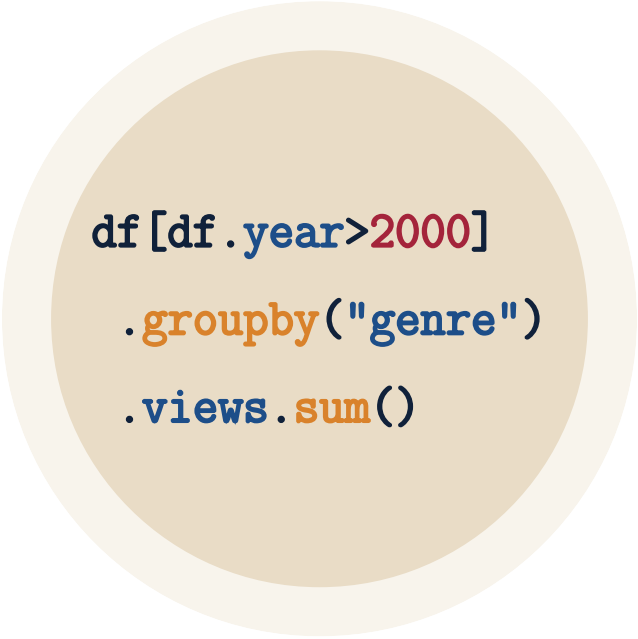
Таблицы в pandas

Теория



Всё, что нужно к задачам на таблицы: прочитать файл, увидеть его ловушки, почистить, посчитать по группам, соединить таблицы, сравнить соседние строки и сдать ответ. Дальше — за подробностями — раздел о pandas в учебнике по анализу данных Яндекса и документация pandas.

Чтение

Таблица — это `DataFrame`: строки с номерами слева и столбцы с названиями. Примеры листка — на трёх маленьких файлах, каждый напечатан целиком. Код идёт подряд, как ячейки ноутбука: что объявлено выше, тем пользуются ниже. Под кодом, с чертой слева, — ровно то, что он печатает. Первый файл, `films.csv`:

```
title,genre,year,rating,views
Северный ветер,драма,1998,7.4,120
Город,комедия,2004,6.9,340
Лето, Драма,2011,8.0,95
Мост,триллер,2004,7.1,210
Остров,драма,2006,,60
Свет,комедия,2015,н/д,410
Река,триллер,2019,7.8,150
Сад,драма,2021,6.5,80
```

```
import pandas as pd
pd.set_option("display.width", 52)
df = pd.read_csv("films.csv")
print(df.head(3))
print(df.shape)
print(df.dtypes)
```

```
             title    genre  year rating  views
0  Северный ветер    драма  1998    7.4    120
1           Город  комедия  2004    6.9    340
2            Лето    Драма  2011    8.0     95
(8, 5)
```

```
title       str
genre       str
year      int64
rating      str
views     int64
dtype: object
```

`head(3)` — первые три строки, `shape` — сколько строк и столбцов, `dtypes` — тип каждого столбца. Тип `str` у `rating` — первый сигнал, что в числах затесался текст.

Если файл прочитался не так, нужен параметр `read_csv`:

- всё в одном столбце — другой разделитель, `sep=";"`;
- числа с запятой, `350,5` — `decimal=","`;
- `UnicodeDecodeError` — файл из Excel, `encoding="cp1251"`; знак `\ufeff` в названии первого столбца — `encoding="utf-8-sig"`;
- код с ведущими нулями, `0045`, прочитается числом `45` — такой столбец читают строкой: `dtype={"kod": str}`;
- разряды через пробел, `1 299` — `thousands=" "` или чистка руками, как ниже.

Сначала посмотреть

Прежде чем считать, смотрят на каждый столбец: какие в нём значения и сколько пропусков. Ловушки таблицы видны здесь, а не в ответе.

```
print(df["genre"].value_counts())
print(df["rating"].unique())
print(df.isna().sum().tolist())
```

```
genre
драма      3
комедия    2
триллер    2
 Драма     1
Name: count, dtype: int64
<StringArray>
['7.4', '6.9', '8.0', '7.1',   nan, 'н/д', '7.8',
 '6.5']
Length: 8, dtype: str
[0, 0, 0, 1, 0]
```

`value_counts()` — сколько раз встречается каждое значение; у жанра их четыре, хотя жанров три: `Драма` с пробелом и заглавной буквой — отдельное значение. `unique()` — все различные значения: в оценках нашлось `н/д`. `isna().sum()` — пропусков по столбцам. Для числового столбца ещё `describe()`: минимум и максимум сразу покажут код ошибки вроде -999.

Чистка

Каждая команда чистки возвращает новый столбец, а не меняет старый, — результат присваивают обратно.

```
df["rating"] = pd.to_numeric(df["rating"],
                             errors="coerce")
df["genre"] = df["genre"].str.strip().str.lower()
print(df["rating"].tolist())
print(df["genre"].value_counts().to_dict())
```

```
[7.4, 6.9, 8.0, 7.1, nan, nan, 7.8, 6.5]
{'драма': 4, 'комедия': 2, 'триллер': 2}
```

`to_numeric(..., errors="coerce")` делает пропуском всё, что не число. Строковые методы — через `.str`: `strip()` срезает пробелы по краям, `lower()` и `upper()` меняют регистр, `replace(a, b)` заменяет кусок строки. Если в ячейке число вместе с текстом, его вынимают регулярным выражением:

```
s = pd.Series(["1,5 км", "800 м", " 2 км", "н/д"])
p = s.str.extract(r"([\d,]+)\s*(км|м)")
print(p)
km = (p[0].str.replace(",", ".").astype(float)
      * p[1].map({"км": 1, "м": 0.001}))
print(km.tolist())
```

```
     0    1
0  1,5   км
1  800    м
2    2   км
3  NaN  NaN
[1.5, 0.8, 2.0, nan]
```

`str.extract` раскладывает строку по скобкам выражения: `[\d,]+` — цифры и запятые, `\s*` — пробелы, `(км|м)` — одно из слов. Не подошедшая строка даёт пропуск. `map` заменяет значения по словарю. Что ещё бывает нужно:

- `s.replace(-999, float("nan"))` — код ошибки в пропуск;
- `df.dropna(subset=["rating"])` — выбросить строки с пропуском, `fillna(0)` — заполнить;
- `df.drop_duplicates()` — убрать полностью одинаковые строки;
- похожие буквы: латинская `A` и русская `А` на вид одинаковы, а для pandas разные, — `value_counts()` покажет оба варианта.

Выбор строк

Условие — в квадратных скобках после таблицы. Каждое сравнение — в круглых, «и» — `&`, «или» — `|`, «не» — `~`:

```
d = df[(df["genre"] == "драма")
       & (df["year"] >= 2000)]
print(d[["title", "year", "rating"]])
print(df.loc[df["views"] > 200, "title"].tolist())
```

```
    title  year  rating
2    Лето  2011     8.0
4  Остров  2006     NaN
7     Сад  2021     6.5
['Город', 'Мост', 'Свет']
```

`df.loc[условие, "столбец"]` — строки по условию и один столбец. Ещё `between(a, b)` — оба конца входят, `isin([...])` — одно из значений, `isna()` — пропуск. Сравнение с пропуском даёт `False`: такая строка не попадает ни в «больше», ни в «меньше».

Счёт и группы

```
v = df["views"]
print(v.sum(), v.mean(), v.median(), v.max())
print(df["rating"].mean(), df["rating"].count())
print(df.loc[v.idxmax(), "title"])
```

```
1465 183.125 135.0 410
7.283333333333334 6
Свет
```

Средняя оценка — по шести фильмам: пропуски не считаются, и `count()` их не включает. `idxmax()` — номер строки с наибольшим значением. Ещё `nunique()` — сколько различных, `mode()[0]` — самое частое.

`groupby` делит строки по значению столбца и считает по каждой группе отдельно; `agg` считает несколько величин сразу:

```
g = df.groupby("genre").agg(n=("title", "size"),
                            views=("views", "sum"),
                            rating=("rating", "mean"))
print(g.round(2))
```

```
          n  views  rating
genre
драма     4    355    7.30
комедия   2    750    6.90
триллер   2    360    7.45
```

Порядок с ничьими. Сортировка по нескольким столбцам: сначала по первому, при равенстве — по второму. Направление — своё у каждого:

```
t = df.sort_values(["year", "views"],
                   ascending=[True, False])
print(t[["title", "year", "views"]].head(4))
```

```
             title  year  views
0  Северный ветер  1998    120
1           Город  2004    340
3            Мост  2004    210
4          Остров  2006     60
```

Так задают правила вроде «больше очков, при равенстве — меньше штраф, затем по алфавиту». Номер места после такой сортировки — `range(1, len(t) + 1)`.

Значение группы в каждой строке

`groupby(...).sum()` сворачивает группу в одно число. Когда строку надо сравнить с её группой, нужен `transform`: столбец той же длины, что таблица, где в каждой строке стоит итог её группы.

```
tot = df.groupby("genre")["views"].transform("sum")
df["share"] = (df["views"] / tot).round(2)
m = df.groupby("genre")["rating"].transform("mean")
print(df[["title", "genre", "share"]].head(4))
print(df.loc[df["rating"] > m, "title"].tolist())
```

```
             title    genre  share
0  Северный ветер    драма   0.34
1           Город  комедия   0.45
2            Лето    драма   0.27
3            Мост  триллер   0.58
['Северный ветер', 'Лето', 'Река']
```

Доля фильма в просмотрах своего жанра и фильмы с оценкой выше средней по жанру. Так же заполняют пропуск средним своей группы: `df["rating"].fillna(m)`.

Соединение таблиц

Второй файл, `bilety.csv`, — сколько билетов продано по дням. Жанра в нём нет, он в `films.csv`; таблицы соединяют по общему столбцу.

```
data,title,sold
2026-10-01,Город,120
2026-10-01,Свет,95
2026-10-02,Город,140
2026-10-02,свет,80
2026-10-03,Город,90
2026-10-04,Город,150
2026-10-04,Свет,110
2026-10-05,Город,130
2026-10-05,Сад,40
```

```
b = pd.read_csv("bilety.csv", parse_dates=["data"])
j = b.merge(df[["title", "genre"]], on="title",
            how="left")
print(j[j["genre"].isna()])
```

```
        data title  sold genre
3 2026-10-02  свет    80   NaN
```

`merge(..., on="title")` ставит рядом строки с одинаковым значением ключа. `how="left"` — все строки левой таблицы остаются, даже без пары; у них справа пропуск. Без него, `how="inner"`, такие строки молча пропали бы. Здесь пары не нашлось из-за `свет` со строчной: ключи сначала приводят к одному виду.

```
key = df["title"].str.lower()
b["key"] = b["title"].str.lower()
j = b.merge(df.assign(key=key)[["key", "genre"]],
            on="key", how="left")
print(j.groupby("genre")["sold"].sum())
```

```
genre
драма       40
комедия    915
Name: sold, dtype: int64
```

Ключи должны совпадать и по типу: число 7 и строка `"7"` или `"T007"` не соединятся — сначала приводят. Строки одной таблицы, у которых нет пары в другой, — `~a["k"].isin(b["k"])`.

Повторы и первые

`drop_duplicates` без параметров убирает полностью одинаковые строки, с `subset` — одинаковые по части столбцов. Какая из одинаковых останется, решает `keep`: первая или последняя *в текущем порядке*. Поэтому сначала сортируют:

```
first = (b.sort_values("data")
         .drop_duplicates(subset="key", keep="first"))
print(first[["key", "data", "sold"]])
```

```
     key       data  sold
0  город 2026-10-01   120
1   свет 2026-10-01    95
8    сад 2026-10-05    40
```

Первая продажа каждого фильма. С `keep="last"` — последняя; так берут, например, последнюю версию записи. То же для одного числа — `b.groupby("key")["data"].min()`.

Сдвиги и окна

Соседние строки сравнивают сдвигом: `shift(1)` — значение из предыдущей строки, `diff()` — разность с ней. Внутри группы — через `groupby`, иначе сдвиг перескочит из одного фильма в другой. Строки сначала упорядочивают:

```
b = b.sort_values(["key", "data"])
gs = b.groupby("key")["sold"]
b["prev"] = gs.shift(1)
b["roll2"] = gs.transform(
    lambda s: s.rolling(2, min_periods=2).mean())
print(b[["key", "data", "sold", "prev", "roll2"]])
```

```
     key       data  sold   prev  roll2
0  город 2026-10-01   120    NaN    NaN
2  город 2026-10-02   140  120.0  130.0
4  город 2026-10-03    90  140.0  115.0
5  город 2026-10-04   150   90.0  120.0
7  город 2026-10-05   130  150.0  140.0
8    сад 2026-10-05    40    NaN    NaN
1   свет 2026-10-01    95    NaN    NaN
3   свет 2026-10-02    80   95.0   87.5
6   свет 2026-10-04   110   80.0   95.0
```

`rolling(2).mean()` — среднее по окну из двух строк, текущей и предыдущей; `min_periods` — сколько значений нужно, чтобы окно считалось. В первой строке группы предыдущей нет — пропуск.

Окно по строкам и окно по времени — разные вещи. У «Света» нет продаж 3 октября, и окно из двух строк 4 октября взяло 2 октября. Окно «за два дня» задают временем, а таблицу индексируют датой:

```
s = b[b["key"] == "свет"].set_index("data")["sold"]
r = s.rolling("2D")
print(pd.DataFrame({"n": r.count(),
                    "mean": r.mean()}))
```

```
                n   mean
data
2026-10-01  1.0   95.0
2026-10-02  2.0   87.5
2026-10-04  1.0  110.0
```

Окно `"2D"` — последние двое суток: `(t - 2 дня, t]`. 4 октября в нём одна продажа, и `count()` это показывает: полное окно отбирают условием на счётчик. Шаги: `"min"`, `"h"`, `"D"`, например `"60min"`. Сдвиг на время, а не на строку, — `shift(freq="1D")`.

Даты и время

Время читают как время, а не как текст: иначе его не сравнить и не вычесть. Файл `log.csv` — журнал действий на сайте:

```
user,time,action
anna,2026-10-19 09:58,вход
boris,2026-10-19 10:05,вход
anna,2026-10-19 10:20,задача
anna,2026-10-19 11:30,задача
boris,2026-10-19 10:40,задача
anna,2026-10-20 08:15,вход
boris,2026-10-20 23:50,задача
anna,2026-10-20 08:40,задача
```

```
log = pd.read_csv("log.csv", parse_dates=["time"])
log = log.sort_values(["user", "time"])
t = log["time"]
print(t.dt.hour.tolist())
print(t.dt.dayofweek.tolist())
print(t.max() - t.min())
```

```
[9, 10, 11, 8, 8, 10, 10, 23]
[0, 0, 0, 1, 1, 0, 0, 1]
1 days 13:52:00
```

`parse_dates` делает столбец временем. Части — через `.dt`: `date` — дата без времени, `hour`, `minute`, `dayofweek` — день недели, понедельник — 0. Разность двух моментов — промежуток, `Timedelta`; `total_seconds()` переводит его в секунды. Сравнивают со строкой: `log[t >= "2026-10-20"]`.

Дата не в стандартном виде разбирается с форматом; если в столбце два формата, разбирают дважды и склеивают:

```
s = pd.Series(["2026-10-19", "20.10.2026"])
d1 = pd.to_datetime(s, format="%Y-%m-%d",
```

```
                      errors="coerce")
d2 = pd.to_datetime(s, format="%d.%m.%Y",
                      errors="coerce")
print(d1.fillna(d2).dt.strftime("%Y-%m-%d").tolist())
td = pd.to_timedelta(["0:05:07", "1:00:30"])
print(td.total_seconds().tolist())
```

```
['2026-10-19', '2026-10-20']
[307.0, 3630.0]
```

`%d` — день, `%m` — месяц, `%Y` — год, `%H:%M` — часы и минуты; не подошедшее `errors="coerce"` делает пропуском, а `fillna` берёт значение из второго разбора. `to_timedelta` читает промежутки вида `ч:мм:сс`.

Ответ

Округлять — самому. `round` в Python округляет половину к чётному, а не вверх, как в школе, а дробь в компьютере хранится не точно:

```
print(round(2.5), round(3.5), round(0.125, 2))
print(f"{df['rating'].mean():.6f}")
```

```
2 4 0.12
7.283333
```

Поэтому число печатают с запасом знаков и округляют по условию руками.

Ответ файлом — без номеров строк, с заголовком как в условии:

```
res = j.groupby("genre")["sold"].sum().reset_index()
res.to_csv("answer.csv", index=False)
print(open("answer.csv", encoding="utf-8").read())
```

```
genre,sold
драма,40
комедия,915
```

`reset_index()` превращает результат группировки обратно в таблицу из двух столбцов, `index=False` не пишет номера строк.